ClubHub

Student Club Management & Discovery Platform

CS310 Mobile Application Development • Final Project Report • Spring 2025-2026

Group Members

Aylin Gül Aksu – 32538
Özge Durmuş – 35483
Sena Yolcu – 22626
Eldar Ismayilzada – 36222
Tural Abdulla – 33736
Ceyda Balcı – 31259

Sabancı University • May 2026

1. Application Overview & Objectives

ClubHub is a Flutter-based mobile application designed to solve a real problem at Sabancı University: the lack of a unified platform through which all student clubs can communicate with and reach students. Today, club announcements are scattered across WhatsApp groups, Instagram pages, and physical noticeboards, making it hard for students to stay informed or discover new clubs.

ClubHub brings every club and student together in one place by providing:

- A personalized social feed of club posts, announcements, and event updates.
- An interactive calendar showing upcoming events filtered to the student's interests.
- Role-based accounts distinguishing students from club board members.
- Real-time data synchronization backed by Firebase Firestore.
- A smooth onboarding flow where students declare their interests to receive tailored content.

The application was developed incrementally over five project steps throughout the Spring 2025-2026 semester, culminating in a fully integrated, tested, and documented product.

2. Implemented Features

2.1 Authentication & Onboarding

- Email/password registration with role selection (Student / Club).
- Login with password reset via email.
- Interest selection screen shown to students after first login (onboarding flow).
- AuthWrapper intelligently routes users to login, onboarding, or home based on auth state.

2.2 Home Feed

- Real-time post stream fetched from Firestore, displayed as a social-media-style card feed.
- Like / unlike posts with immediate UI update (optimistic update via provider).
- Post search via a dedicated search delegate.
- Stories bar at the top for club activity highlights.
- Support for dark mode with full theme switching.

2.3 Post Management (Club Accounts)

- Create posts with optional caption, location, and image.
- Edit existing posts (caption & location).
- Delete posts.
- Image upload via image_picker package.

2.4 Comments

- Threaded comment section per post.
- Real-time comment loading from Firestore.
- Add and delete comments.

2.5 Events & Calendar

- Club accounts can create events with title, date, time, and attendance status.
- Interactive calendar screen showing events per day.
- Day detail screen listing all events for a selected date.
- Events streamed in real-time per user from Firestore.

2.6 Profiles

- Student profile screen showing bio, interests, and avatar.
- Club profile screen showing club information and posts.
- Profile photo picker backed by image_picker.

2.7 Settings & Theme

- Dark/light mode toggle persisted via SharedPreferences.
- Settings screen accessible from the navigation drawer.

2.8 Navigation

- Bottom navigation bar for quick access to Home, Calendar, and Profile.
- Side drawer for extended navigation options and settings.
- Named routes throughout the application for clean navigation.

3. Firebase Usage

3.1 Firebase Authentication

Firebase Authentication handles all identity management in ClubHub. The AuthService class wraps the FirebaseAuth SDK and exposes clean async methods consumed by the AuthProvider.

- signUp(): Creates a FirebaseAuth user, then writes a UserModel document to Firestore.
- signIn(): Email/password sign-in with structured error handling.
- signOut(): Clears local state and routes back to the login screen.
- resetPassword(): Sends a Firebase password reset email.
- authStateChanges stream: The AuthProvider listens continuously so the UI reacts instantly to auth state changes without polling.

Role information (student vs. club) is stored alongside user data in Firestore and read back on every login, allowing the app to conditionally show club-only features such as post creation and event management.

3.2 Cloud Firestore

Firestore is the primary database. All read operations use real-time snapshots (.snapshots()) so the UI updates live without requiring manual refresh.

Collections used:

- users – UserModel documents (id, email, username, role, interests, bio, avatarUrl, onboardingComplete).
- posts – PostModel documents (clubName, caption, imageUrl, location, likes[], createdBy, createdAt).
- events – EventModel documents (title, clubName, date, time, status, createdBy, createdAt).
- comments – CommentModel documents scoped to a post (text, authorId, createdAt).

Security: Firestore reads and writes are mediated through service classes (PostService, EventService, AuthService, etc.), which centralise query logic and make it easy to add Firestore security rules in a future iteration.

3.3 Firebase Storage

Firebase Storage is configured and referenced in the project for future media uploads (post images and profile photos). In the current version, the image_picker package is used to select images on-device, and the upload path to Firebase Storage is prepared in the service layer for integration.

3.4 Firebase Configuration

The project uses the FlutterFire CLI-generated firebase_options.dart for platform-specific configuration (Android google-services.json included). Firebase is initialized before runApp() to ensure all services are ready before the first frame is rendered.

4. State Management

ClubHub uses the Provider package for state management, following the ChangeNotifier pattern. This approach was chosen for its simplicity, tight Flutter integration, and suitability for a project of this scale.

Providers registered at the app root via MultiProvider:

- AuthProvider – Manages authentication state (AuthStatus enum), the current UserModel, loading flags, and error messages. Listens to Firebase auth state changes.
- PostProvider – Manages the posts list, loading state, and CRUD operations delegated to PostService.
- EventProvider – Manages the events list with real-time Firestore streaming, user-scoped queries, and helper methods such as eventsForDay().
- CommentProvider – Manages comments for the currently viewed post.

- ThemeProvider – Manages dark/light mode toggle, persisting the preference via SharedPreferences through PreferencesService.

Widgets consume providers through context.watch<T>() (reactive rebuilds) and context.read<T>() (one-shot calls, e.g. inside callbacks), following Flutter best practices to minimise unnecessary rebuilds.

5. Application Screenshots

The following screens represent the key user journeys in ClubHub. Screenshots are available in the project's video demonstration.

- Login & Registration – Email/password forms with role selection and password reset dialog.
- Interest Selection – Onboarding screen where students pick their club interest categories.
- Home Feed – Card-based post feed with like button, club name, caption, and comment access.
- Stories Bar – Horizontal scrollable bar above the feed showing club activity indicators.
- Create Post – Form for club accounts to add caption, location, and image.
- Comments Screen – Threaded comment list with add/delete functionality.
- Calendar Screen – Monthly calendar view with event dots on active days.
- Day Detail – List of events for a selected calendar day with time and status.
- Add Event – Form for club accounts to schedule a new event.
- Student Profile – User bio, interests, and avatar display.
- Club Profile – Club description and published posts.
- Settings – Dark mode toggle and account options.

6. Testing

6.1 Testing Strategy

Testing was led by Sena Yolcu and Ceyda Balcı. The goal was to validate critical business logic and widget rendering without requiring a live Firebase connection, keeping tests fast and deterministic.

6.2 Unit Tests

Unit tests target model and provider logic in isolation:

- UserModel serialization – Verifies that toFirestore() produces the correct map structure and that fromFirestore() correctly parses all fields including the optional avatarUrl.
- PostModel serialization – Validates round-trip conversion between PostModel and Firestore maps, including Timestamp ↔ DateTime conversion.

- EventProvider.isSameDay() – Confirms that the helper returns true for identical dates and false for different days.
- ThemeProvider.toggleTheme() – Verifies that the isDarkMode flag flips correctly after calling toggleTheme().

6.3 Widget Tests

Widget tests validate that key UI components render correctly and respond to user interaction:

- Login screen – Confirms that the email field, password field, and login button are present in the widget tree.
- Interest selection screen – Verifies that interest category chips are rendered and tappable.
- Home feed card – Tests that a PostModel renders its clubName and caption correctly inside a Card widget.

6.4 Running Tests

All tests are executed with the standard Flutter command:

```
flutter test
```

All tests pass successfully with zero errors or warnings.

7. Challenges & Solutions

Challenge 1: Firebase Auth State Race Condition

Problem: On cold start the AuthWrapper sometimes rendered the login screen for a fraction of a second before recognising an already-logged-in user, causing a flash of wrong content.

Solution: A 5-second safety timer was added to AuthProvider. If Firebase has not responded within 5 seconds, the state defaults to unauthenticated. A loading splash screen is shown while AuthStatus.unknown is active.

Challenge 2: Firestore Composite Index Requirement

Problem: Queries combining where() and orderBy() on different fields required a composite Firestore index that was not automatically created, causing runtime exceptions.

Solution: Sorting was moved from Firestore queries to Dart-side list sorting after the snapshot arrives (e.g. list.sort((a, b) => a.date.compareTo(b.date))). This eliminates the composite index requirement while keeping the query simple.

Challenge 3: Named Route Arguments

Problem: The original /comments route used a typo (postI instead of postId) and did not extract arguments from ModalRoute, causing a crash when navigating to the comments screen.

Solution: The route handler was refactored to extract arguments via ModalRoute.of(context)?.settings.arguments and cast them safely with a fallback value.

Challenge 4: Dark Mode Consistency

Problem: Not all screens respected the global theme; some used hardcoded colours, leading to illegible text in dark mode.

Solution: A ThemeProvider was introduced and all screens were updated to derive their colour values from context.watch<ThemeProvider>().isDarkMode rather than using hardcoded hex values.

Challenge 5: State Leak on Logout

Problem: Post and event streams continued running after a user signed out, sometimes displaying previous user's data on re-login.

Solution: EventProvider.dispose() cancels the Firestore stream subscription. On sign-out, AuthProvider clears the UserModel reference, and the EventProvider re-initialises its subscription when a new userId is passed to listenToEvents().

8. Lessons Learned

- Incremental development works: Building the app step-by-step across five milestones helped each member absorb Flutter concepts at a manageable pace and kept integration problems small.
- Provider is a strong fit for mid-size Flutter apps: The simplicity of ChangeNotifier + MultiProvider was sufficient to handle all state scenarios we encountered without needing a more complex solution like Bloc or Riverpod.
- Firebase real-time streams require lifecycle management: Forgetting to cancel StreamSubscriptions caused memory leaks and stale data bugs. Properly implementing dispose() in providers is essential.
- Git discipline matters: Meaningful commit messages and feature branches made it far easier to trace which change introduced a bug and to review each member's contributions.
- Writing tests early reveals design flaws: When we attempted to write widget tests for screens tightly coupled to Firebase, we realized the need to abstract service calls behind interfaces. This pushed us toward cleaner architecture.
- Role-based logic adds complexity: Supporting two distinct user roles (student vs. club) in a single app required careful conditional rendering throughout the UI and consistent role checks in service layer calls.

9. Individual Contributions

The table below summarises each team member's primary responsibilities. All members participated in code reviews and testing sessions.

Name	ID	Role	Contributions
Aylin Gül Aksu	32538	Documentation Lead	README, report writing, Firebase config docs

Name	ID	Role	Contributions
Özge Durmuş	35483	Project Coordinator	Sprint planning, screen coordination, UI reviews
Sena Yolcu	22626	QA Lead	Test cases, widget tests, bug reporting
Eldar Ismayilzada	36222	Integration Lead	Firebase integration, repo management, CI
Tural Abdulla	33736	Documentation Lead	Report sections, API documentation, comments
Ceyda Balcı	31259	QA / Research Lead	Unit tests, UX research, accessibility review

10. Repository & Demonstration Links

GitHub Repository: https://github.com/eldarismayilzada-art/CS310-Group-Project.git
Video Demonstration: https://youtu.be/woBSjisgZZA
Flutter Version: 3.11.3 (Dart SDK ^3.11.3)
Firebase Project: clubhub (Android: com.example.clubhub)

11. Known Limitations & Future Work

- Firebase Storage upload is prepared in the service layer but not yet fully wired to the UI; images are selected on-device but not persisted to a remote URL.
- Push notifications for event reminders are planned but not yet implemented.
- The search feature searches post captions in-memory; a Firestore full-text search solution (e.g. Algolia) would improve scalability.
- Club discovery and recommendation algorithms are identified as nice-to-have features and are left for a future version.
- Firestore security rules are not yet configured; production deployment would require proper read/write rules.